Neural Network Framework from scratch in C++

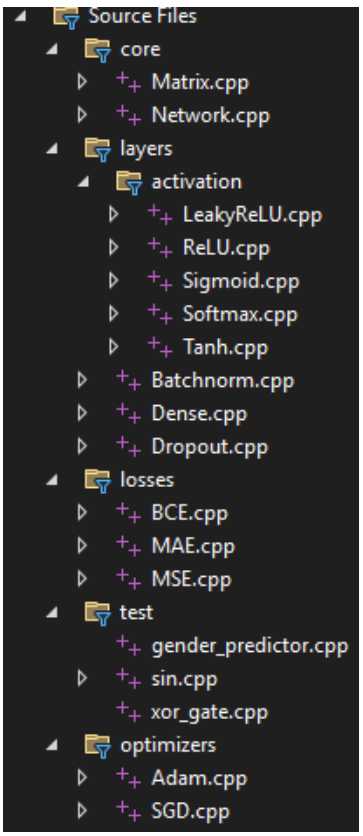
Introduction

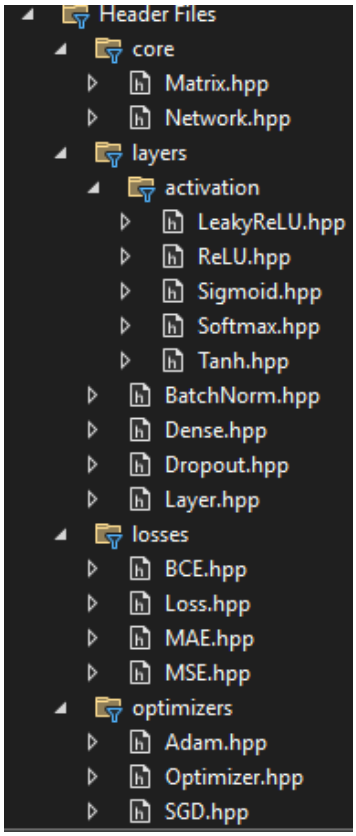
This project implements a neural network framework from scratch that support basic operations such as designing and training custom neural networks.

Programming Language

The progamming language chosen for this project was C++, because of its speed compared to other languages, while also being able to control the memory and system resources.

Features





The projects has the following main components:

Core – contain the logic behind the matrix operations and the network that binds the layers.

Layers – the different types of layers that can be used, as well as the activation layers.

Losses – the loss functions that can be used

Optimizers – neural network optimizers that can be used

Test – three simple test cases for the framework

Test cases

I created three files containing simple neural network architectures to test if the framework works. I chose those simple problems so that the training will be fast and not resource heavy.

XOR Gate

For the first problem, I used the framework to compute the XOR gate.

The architecture used:

```
net.add_layer(std::make_unique<Dense>(2, 4, WeightInit::Glorot));
net.add_layer(std::make_unique<Sigmoid>());
net.add_layer(std::make_unique<Dense>(4, 1, WeightInit::Glorot));
net.add_layer(std::make_unique<Sigmoid>());

net.set_loss(std::make_unique<BCE>());
```

The results:

```
[Epoch 5000/5000] Avg Loss: 0.0278803

===== XOR RESULTS =====
0 XOR 0 = 0 | pred=0.0152649 OK
0 XOR 1 = 1 | pred=0.977604 OK
1 XOR 0 = 1 | pred=0.978693 OK
1 XOR 1 = 0 | pred=0.0505447 OK

Accuracy: 4/4 (100%)

===== TEST CASES =====
0 XOR 0 -> 0.0152649
0 XOR 1 -> 0.977604
1 XOR 0 -> 0.978693
1 XOR 1 -> 0.0505447
```

Sin(x)

For the second problem, I tried estimating the sine function using two simple architectures.

The architectures used:

V1

```
net.add_layer(std::make_unique<Dense>(1, 16, WeightInit::Glorot));
net.add_layer(std::make_unique<Sigmoid>());
net.add_layer(std::make_unique<Dense>(16, 16, WeightInit::Glorot));
net.add_layer(std::make_unique<Sigmoid>());
net.add_layer(std::make_unique<Dense>(16, 1, WeightInit::Glorot));

net.set_loss(std::make_unique<MSE>());
```

V2

```
net.add_layer(std::make_unique<Dense>(1, 16, WeightInit::Glorot));
net.add_layer(std::make_unique<Tanh>());
net.add_layer(std::make_unique<Dense>(16, 16, WeightInit::Glorot));
net.add_layer(std::make_unique<Tanh>());
net.add_layer(std::make_unique<Dense>(16, 1, WeightInit::Glorot));

net.set_loss(std::make_unique<MSE>());

Adam optimizer(0.001);
```

The results:

V1

```
[Epoch 10000/10000] Avg Loss: 5.04865e-05

===== SIN(x) APPROXIMATION =====
x=0 sin(x)=0 pred=0.00624669 error=0.00624669
x=0.628319 sin(x)=0.587785 pred=0.587194 error=0.000591493
x=1.25664 sin(x)=0.951057 pred=0.948032 error=0.00302458
x=1.88496 sin(x)=0.951057 pred=0.945098 error=0.0059583
x=2.51327 sin(x)=0.587785 pred=0.586251 error=0.00153397
x=3.14159 sin(x)=1.22465e-16 pred=-0.00616133 error=0.00616133
x=3.76991 sin(x)=-0.587785 pred=-0.586374 error=0.00141155
x=4.39823 sin(x)=-0.951057 pred=-0.941755 error=0.00930118
x=5.02655 sin(x)=-0.951057 pred=-0.957661 error=0.00660403
x=5.65487 sin(x)=-0.587785 pred=-0.573943 error=0.0138421

===== TEST CASES =====
x=0 sin(x)=0 pred=0.00624669
x=1.5708 sin(x)=1 pred=0.993078
x=3.14159 sin(x)=1.22465e-16 pred=-0.00616133
x=4.71239 sin(x)=-1 pred=-1.00671
x=6.28319 sin(x)=-2.44929e-16 pred=-0.0316122
```

V2

```
[Epoch 10000/10000] Avg Loss: 0.000113195

===== SIN(x) APPROXIMATION =====
x=0 sin(x)=0 pred=0.0119272 error=0.0119272
x=0.628319 sin(x)=0.587785 pred=0.598547 error=0.0107622
x=1.25664 sin(x)=0.951057 pred=0.959903 error=0.00884669
x=1.88496 sin(x)=0.951057 pred=0.966721 error=0.0156648
x=2.51327 sin(x)=0.587785 pred=0.59895 error=0.0111645
x=3.14159 sin(x)=1.22465e-16 pred=0.0172173 error=0.0172173
x=3.76991 sin(x)=-0.587785 pred=-0.574924 error=0.0128614
x=4.39823 sin(x)=-0.951057 pred=-0.930755 error=0.0203013
x=5.02655 sin(x)=-0.951057 pred=-0.94365 error=0.00740667
x=5.65487 sin(x)=-0.587785 pred=-0.574351 error=0.0134342

===== TEST CASES =====
x=0 sin(x)=0 pred=0.0119272
x=1.5708 sin(x)=1 pred=1.01174
x=3.14159 sin(x)=1.22465e-16 pred=0.0172173
x=4.71239 sin(x)=-1 pred=-0.988141
x=6.28319 sin(x)=-2.44929e-16 pred=-0.0174161
```

Gender predictor based on height and weight

For the third example, I used a simple dataset to predict the gender based on the samples height and weight.

The architecture used:

```
test.add_layer(std::make_unique<Dense>(2, 16, WeightInit::Glorot));
test.add_layer(std::make_unique<ReLU>());
test.add_layer(std::make_unique<Dropout>(0.2));
test.add_layer(std::make_unique<Dense>(16, 1, WeightInit::Glorot));
test.add_layer(std::make_unique<Sigmoid>());
test.set_loss(std::make_unique<BCE>());

Adam optimizer(0.001);
```

The results:

```
[Epoch 2000/2000] Avg Loss: 0.139957

===== TRAINING SET =====
h=181 w=77 | label=1 pred=0.90881 OK
h=176 w=90 | label=1 pred=0.984215 OK
h=177 w=75 | label=1 pred=0.847429 OK
h=188 w=90 | label=1 pred=0.992512 OK
h=195 w=90 | label=1 pred=0.995165 OK
h=183 w=82 | label=1 pred=0.96324 OK
h=179 w=78 | label=1 pred=0.912462 OK
h=192 w=95 | label=1 pred=0.997217 OK
h=186 w=88 | label=1 pred=0.988604 OK
h=174 w=71 | label=1 pred=0.701535 OK
h=180 w=80 | label=1 pred=0.939524 OK
h=187 w=85 | label=1 pred=0.982348 OK
h=191 w=92 | label=1 pred=0.99538 OK
h=178 w=76 | label=1 pred=0.874903 OK
h=185 w=84 | label=1 pred=0.976496 OK
h=149 w=44 | label=0 pred=0.0591245 OK
h=160 w=45 | label=0 pred=0.0644264 OK
h=154 w=54 | label=0 pred=0.0829818 OK
h=171 w=54 | label=0 pred=0.102415 OK
h=176 w=52 | label=0 pred=0.104071 OK
h=158 w=51 | label=0 pred=0.0770542 OK
h=163 w=57 | label=0 pred=0.1046 OK
h=155 w=48 | label=0 pred=0.0691175 OK
h=168 w=61 | label=0 pred=0.231033 OK
h=172 w=63 | label=0 pred=0.350265 OK
h=157 w=50 | label=0 pred=0.0743199 OK
h=161 w=55 | label=0 pred=0.0885109 OK
h=165 w=58 | label=0 pred=0.130806 OK
h=153 w=47 | label=0 pred=0.066319 OK
h=169 w=60 | label=0 pred=0.212615 OK

Training accuracy: 30/30 (100%)

===== TEST CASES =====
185cm 88kg  (male)   -> 0.987886
150cm 48kg  (female) -> 0.0673579
170cm 65kg  (unsure) -> 0.400258
198cm 82kg  (male)   -> 0.985134
162cm 57kg  (female) -> 0.100753
175cm 70kg  (unsure) -> 0.678715
```